

# Classwork: TypeScript Functions

100 + 10 bonus | VS Code first | Backup: [typescriptrlang.org/play](https://typescriptrlang.org/play)

Student name

Date

## Part 1 - Definition or Call?

10 points

```
function celebrate() {  
  console.log("Mission complete!");  
}  
  
celebrate();  
celebrate();
```

Identify the definition, one call, and the number of outputs:

What would print if both calls were deleted? Why?

Explain defining versus calling in your own words:

### Core rule

A definition describes the job. A call runs the job.

# Define Once, Call Many Times

Part 2 | 15 points

## Part 2 - Reuse One Function

15 points

```
function showWarmUp() {  
  console.log("Open your file.");  
  console.log("Read the goal.");  
  console.log("Predict before running.");  
}
```

**Predict the output before adding a call:**

**Write three calls, then record the complete output:**

**How many times did the body run, and how do you know?**

**Why is this easier to reuse than copied output lines?**

# Parameters and Arguments

Part 3 | 20 points

## Part 3A - One Typed Parameter

12 points

```
function greetPlayer(name: string) {  
  console.log(`Welcome, ${name}!`);  
}  
greetPlayer("Maya");  
greetPlayer("Leo");
```

Name the function, parameter, type, and arguments. Predict both outputs:

Write a third call and explain why `greetPlayer(42)` is rejected:

## Part 3B - Two Typed Parameters

8 points

```
function reportScore(player: string, score: number) {  
  console.log(`${player} scored ${score} points.`);  
}  
reportScore("Zara", 95);
```

Match by position, add one call, and repair `reportScore(95, "Zara")`:

# A Decision Inside a Function

Part 4 | 20 points

## Part 4 - Reuse a Familiar Decision

20 points

```
function checkScore(score: number) {  
  if (score >= 80) {  
    console.log(`${score}: mission passed`);  
  } else {  
    console.log(`${score}: keep practicing`);  
  }  
}  
checkScore(92);  
checkScore(67);  
checkScore(80);
```

Predict the exact three output lines:

Why does 80 pass? What old skill is reused inside the function?

Add an else-branch call. Change the boundary to 70 and predict again:

# AI Label Review Helper

Part 5A | 25 points total

## Part 5A - Define and Call the Review Rule

12 points

This is a rule-based review helper, not an AI model. A person remains responsible for checking the original example.

```
function reviewLabel(label: string) {  
  if (label === "unknown") {  
    console.log("Human review needed.");  
  } else {  
    console.log(`${label}: label accepted for now.`);  
  }  
}  
reviewLabel("cat");  
reviewLabel("unknown");  
reviewLabel("tree");
```

**Predict all three outputs and identify the review-triggering argument:**

**What prints before any call is added? Explain:**

**Why say accepted for now instead of guaranteed correct?**

# Reuse the AI Helper in a Loop

Part 5B | 13 points

## Part 5B - Call Once Per Array Item

13 points

```
const labels: string[] = ["cat", "unknown", "tree", "unknown"];

for (let i = 0; i < labels.length; i++) {
  reviewLabel(labels[i]);
}
```

Trace each round: i, argument, and output:

| i | argument | output |
|---|----------|--------|
|   |          |        |
|   |          |        |
|   |          |        |
|   |          |        |

How many calls happen? Which arguments request review?

| calls | arguments |
|-------|-----------|
|       |           |
|       |           |
|       |           |
|       |           |

Add one known and one unknown label. Predict the added outputs:

| known | unknown |
|-------|---------|
|       |         |
|       |         |
|       |         |
|       |         |

What must a person inspect before changing a label?

| inspected |
|-----------|
|           |
|           |
|           |
|           |

# Repair the Function Bugs

Part 6 | 10 points

## Bug A - Defined but Never Called

5 points

```
function showReady() {  
  console.log("Ready!");  
}
```

Explain the problem, add the smallest fix, and predict the output:

## Bug B - Arguments in the Wrong Order

5 points

```
function showItem(item: string, quantity: number) {  
  console.log(`${quantity} ${item}`);  
}  
showItem(3, "notebooks");
```

Explain the contract problem, repair the call, and predict the output:

Core score: 100 points | Continue only if return was introduced.

# Bonus: Return a Result

+10 bonus points

## Bonus - A Typed Result Comes Back

+10 points

```
function needsReview(label: string): boolean {  
  if (label === "unknown") {  
    return true;  
  } else {  
    return false;  
  }  
}
```

Predict the returned values for unknown and cat:

Store one result in a typed variable and print it:

Explain the difference between printing and returning:

Write isPassing(score: number): boolean using a boundary of 70:

Send to Ishwari ma'am + Khushi ma'am. Optional: reply to the Teams classwork post.